MongoDB Content Management System

By ARUN PRASATH K (241001018)

Database Setup & Collection Creation

Initialize Database

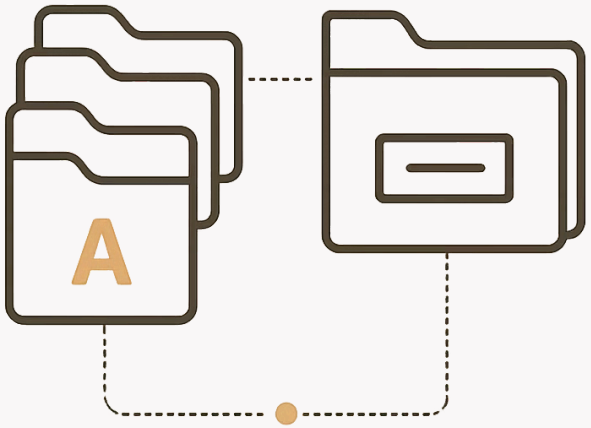
Start by creating and switching to your CMS database. This establishes the foundation for all content storage and management operations.

```
use cmsDB
```

Create Collections

Build collections to store different types of content. Each collection acts as a container for related documents.

```
db.createCollection("users")
db.createCollection("posts")
db.createCollection("comme
nts")
```



Schema Validation: Users Collection

Define strict validation rules to ensure data integrity and consistency across all user documents.

```
db.createCollection("users", {
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["username", "email", "role"],
 properties: {
 username: { bsonType: "string" },
 email: { bsonType: "string" },
 role: { bsonType: "string" },
 age: { bsonType: ["int", "null"] },
 isActive: { bsonType: "bool" }
 }
 }
 }
})
```

Schema Validation: Posts Collection

Required Fields

Title, category, author, and views must be present in every post document.

Enum Constraints

Category limited to Technology, Education, Lifestyle, or News values.

Pattern Matching

Post IDs must follow POST#### format with exactly 4 digits.

```
db.createCollection("posts", {
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["title", "category", "author", "views"],
 properties: {
 title: { bsonType: "string", minLength: 5, maxLength: 150 },
 category: { enum: ["Technology", "Education", "Lifestyle", "News"] },
 author: { bsonType: "string" },
 views: { bsonType: "int", minimum: 0 },
 postId: { bsonType: ["string", "null"], pattern: "^POST[0-9]{4}$" }
 }
 }
 },
 validationLevel: "strict",
 validationAction: "error"
})
```

Insert Operations

Insert Single Document

Add one document at a time with automatic ID generation and validation enforcement.

```
db.users.insertOne({
  username: "arun123",
  email: "arun@gmail.com",
  role: "admin",
  age: 21,
  isActive: true
}) db.posts.insertOne({ title: "MongoDB Basics", author:
"arun123", views: 100 }
```

Insert Multiple Documents

Bulk insert operations for efficient data loading. All documents validated before insertion.

```
db.posts.insertMany([
 {
 title: "First Post",
 category: "Technology",
 author: "admin",
 views: 0
 },
 {
 title: "Second Post",
 category: "Education",
 author: "editor",
 views: 0
 }
])
```

Query Operations: Finding Data

Basic Find

Retrieve all documents from a collection.

```
db.users.find()
```

Find by Field

Query specific documents using field criteria.

```
db.users.find({ username: "admin" })
```

Find with Conditions

Apply filters to narrow search results.

```
db.posts.find({ category: "Technology" })
```

Update Operations

Update One

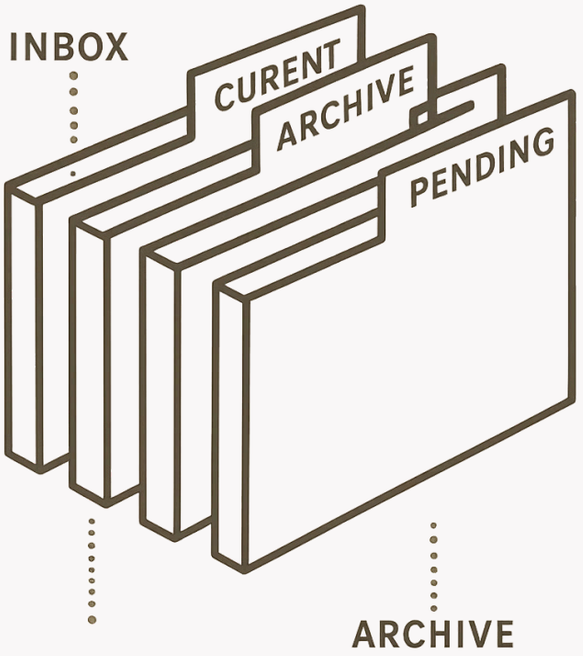
Modify a single document matching the query criteria.

```
db.users.updateOne(
  { username: "admin" },
  { $set: { role: "editor" } }
)
```

Update Many

Apply changes to multiple documents simultaneously.

```
db.posts.updateMany(
  { category: "Technology" },
  { $inc: { views: 1 } }
)
```



Indexing Strategies

Single Field Index

Create indexes on frequently queried fields to dramatically improve search performance.

```
db.users.createIndex({
username: 1 })
db.posts.createIndex({
category: 1 })
```

Compound Index

Build multi-field indexes for complex queries involving multiple criteria.

```
db.posts.createIndex({
  category: 1,
  views: -1
})
```

3

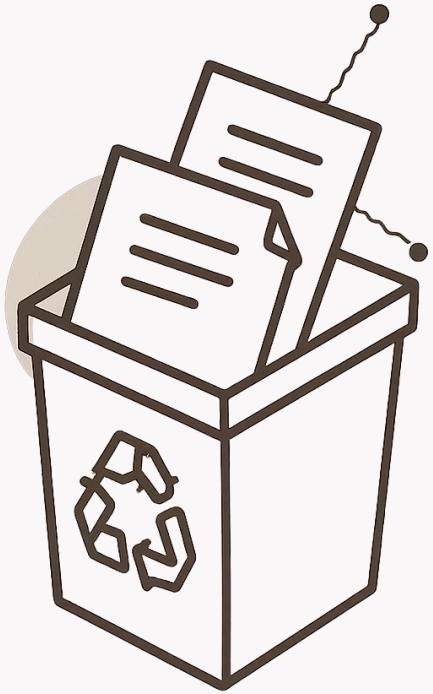
Indexes Created

Users, posts, and compound indexes

100%

Validation

Strict enforcement on all operations



Delete Operations

Delete One Document

Remove a single document matching the specified criteria. Returns deletion count.

```
db.users.deleteOne({ username: "admin" })
```

Delete Multiple Documents

Remove all documents matching the condition. Use with caution as it affects multiple records.

```
db.posts.deleteMany({ views: { $lt: 10 } })
```

Verification & Management Commands

Collection Information

Retrieve metadata and validation rules for any collection.

```
db.getCollectionInfos({ name: "posts" })
```

Index Inspection

View all indexes on a collection to verify optimization strategy.

```
db.users.getIndexes()
db.posts.getIndexes()
```

01

Create collections with validation

02

Insert and validate documents

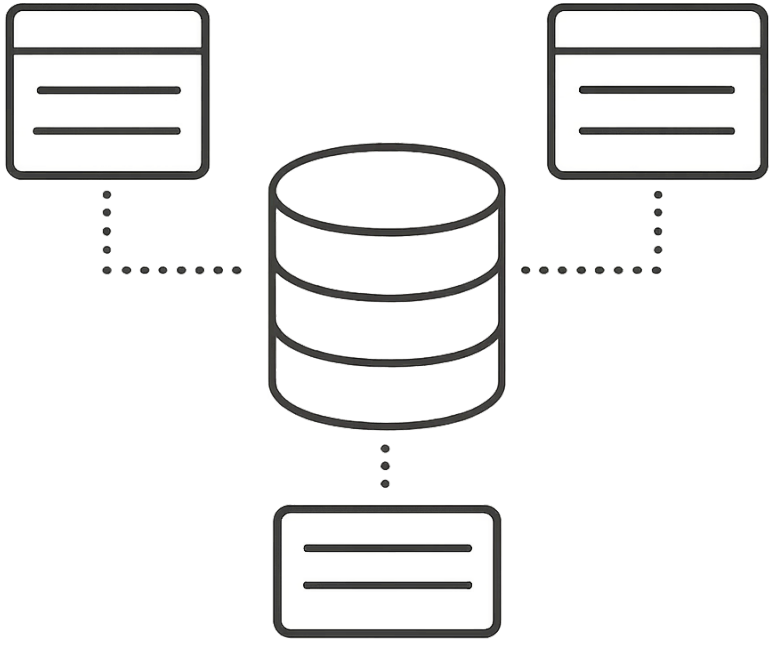
03

Build indexes for performance

04

Query, update, and delete data

Data Modeling & Schema Design



Collection Structure

Design collections to represent different entity types in your CMS:

- Users collection: User accounts and permissions
- Posts collection: Content articles and metadata
- Comments collection: User comments and feedback

Relationships

Use document references (IDs) to establish relationships between collections while maintaining data integrity and enabling efficient queries.

Document Validation & Schema Enforcement

1

Define Required Fields

Specify mandatory fields in the schema that must exist in every document

2

Set Data Types

Enforce BSON data types to ensure data consistency across documents

3

Add Constraints

Implement min/max values, string lengths, patterns, and enum values

4

Configure Validation

Set validation level and action to control strictness and error handling

Visualizing CMS Architecture

Data Storage Layer

MongoDB collections storing users, posts, and comments as JSON documents with flexible schemas

Validation Layer

Schema enforcement ensuring data integrity through required fields, type checking, and constraints

Query Performance

Indexing strategies optimizing read operations and enabling efficient document retrieval

CRUD Operations

Complete create, read, update, and delete capabilities for full content lifecycle management